



Kauno technologijos universitetas

Elektros ir elektronikos fakultetas

T170B008 Kompiuterinės komunikacijos

Jutiklių duomenų vizualizavimas

Laboratorinis darbas Nr. 2

doc. dr. Pranas Kuzas

Simonas Riauka

Aurimas Junevičius

Kaunas, 2025

Turinys

Įvadas	3
1. Python programavimo kalbos bei įrankių įdiegimas	4
1.1. Programavimo paketo instaliavimas	4
1.2. Teksto redaktoriaus instaliavimas	4
2. Grafinė sąsaja Python aplinkoje.....	8
3. Python grafinės sąsajos pagrindiniai funkcionalumai	9
3.1. COM PORT	9
3.1.1. Prievado pasirinkimas	9
3.1.2. Prievado atidarymas ir uždarymas	10
3.2. Grafiko sukūrimas.....	11
3.3. UART duomenų priėmimas.....	12
3.4. UART duomenų apdorojimas.....	12
3.5. Grafinės sąsajos elementų išdėstymas.....	13
4. Laboratorinio darbo užduotis.....	16
5. Kontroliniai klausimai	17
Naudingos nuorodos	18

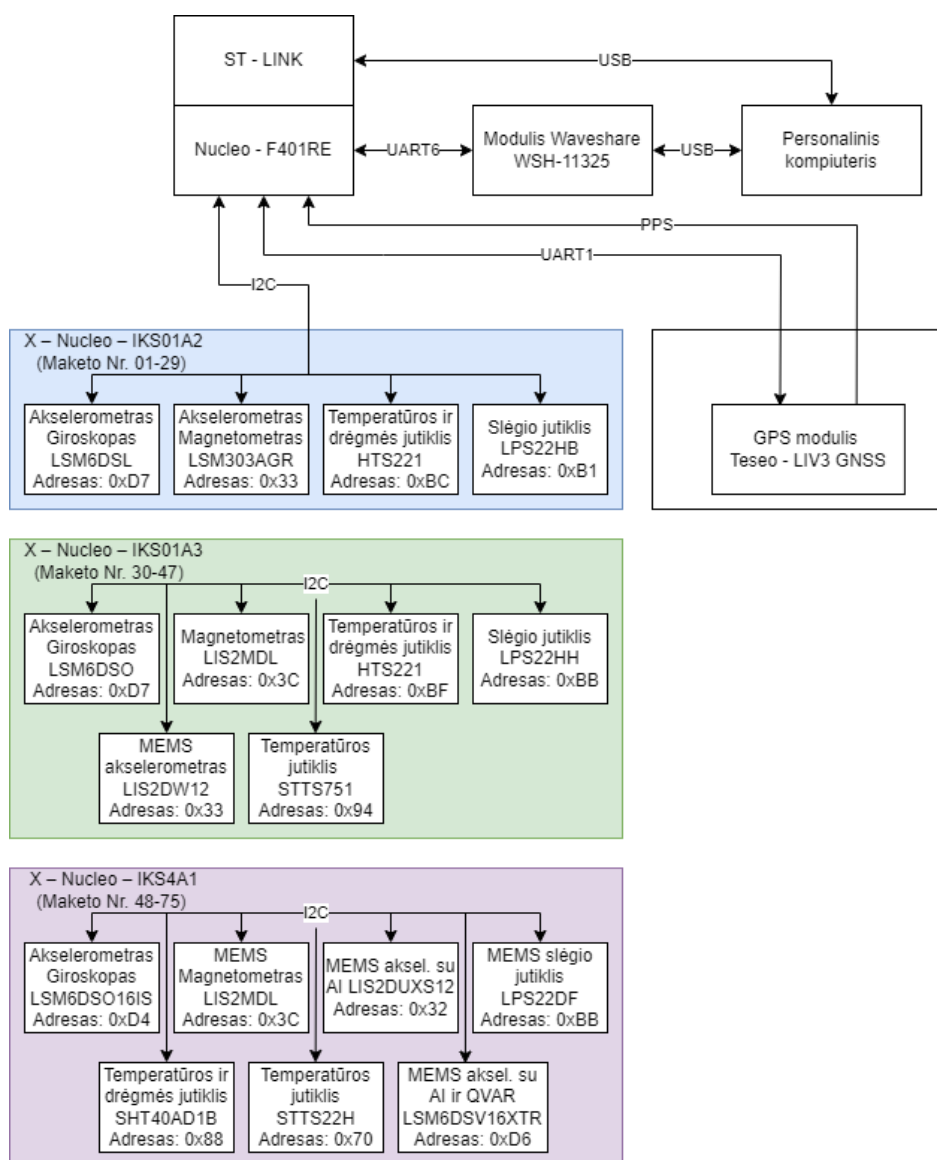
Įvadas

Šiuolaikinės duomenų surinkimo, apdorojimo, testavimo sistemos priima informaciją iš įvairių jutiklių analoginių arba skaitmeninių signalų pavidalu per nuoseklias bei lygiagrečias sąsajas. Šio laboratorinio darbo metu bus siekiama priimti asmeniniame kompiuteryje duomenis, gaunamus iš laboratorinio darbo maketo, ir juos vizualizuoti Python programinėje aplinkoje. Laboratorinio darbo metu naudojama tokia pat įranga, kaip ir pirmajame laboratoriniame darbe, susidedanti iš „STMicroelectronics“ kompanijos produktų. Grafinėje vartotojo sąsajoje atvaizduojamas duomenų paketas suformuotas pagal individualią pirmojo laboratorinio darbo užduotį.

Laboratorinio darbo tikslas:

1. Atvaizduoti per UART sąsają priimamus duomenis susikurtoje grafinėje vartotojo sąsajoje.

Laboratorinio darbo maketo supaprastinta struktūrinė schema pateikiama 1 pav. Į asmeninį kompiuterį duomenys yra perduodami UART6 sąsaja per UART/USB modulį Waveshare WSH-11325 – 11325.



1 pav. Laboratorinio darbo maketo su IKS01A2, IKS01A3 ir IKS4A1 jutiklių moduliais struktūrinė schema.

1. Python programavimo kalbos bei įrankių įdiegimas

Python programavimo kalba sukurta lengvam panaudojimui. Skirtingai nei C ar C++ kalbos, ji nėra kompiliuojama, todėl paleidus programą ji iš karto pradės veikti. Toks veikimas galimas Python interpretatoriaus – jis programos veikimo metu kiekvieną eilutę iš eilės paverčia į kompiuterines komandas ir taip įvykdo reikalingas operacijas. Toks programavimo kalbos realizavimas suteikia paprastumo naudotojui, bet kyla ir problemų – paleidus programą kodo klaida gali būti aptikta tik jai įsibėgėjus, todėl būtina pilnai išbandyti programos veikimą prieš nutariant, kad ji veiks tinkamai.

1.1. Programavimo paketo instaliavimas

Python programavimo paketą atsisiųsti galima iš <https://www.python.org/downloads/>. Atidarius atsisiųstą „.exe“ failą pradedama kalbos paketo instaliacija.

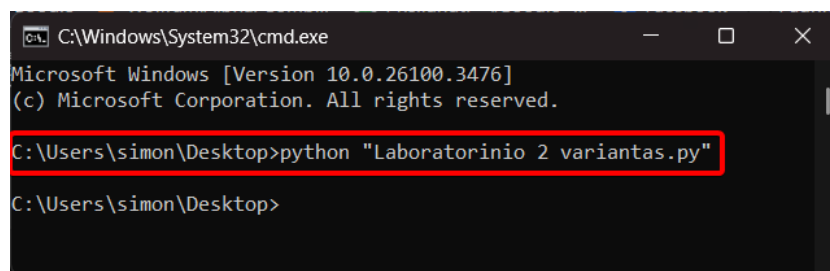
Įsidiegiant Python programinį paketą yra svarbu pažymėti „Add python.exe to PATH“ varnelę, kad Python būtų galima pasiekti iš bet kurio kompiuterio aplankalo. Siūloma rinktis bazinę instaliavimo konfigūraciją paspaudžiant „Install Now“ mygtuką (žr. 2 pav.).



2 pav. Python įrangos įdiegimo langas.

1.2. Teksto redaktoriaus instaliavimas

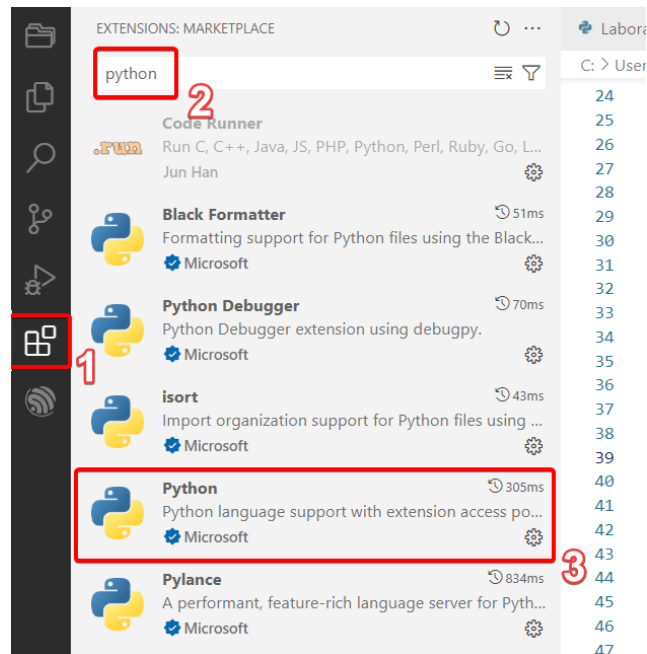
Python galima naudoti be papildomų instaliacijų vien komandinės eilutės (angl. *Command Line*) pagalba. Susikūrus Python failą jį paleisti galima įrašius „python failas.py“ į terminalo langą (žr. 3 pav.).



3 pav. Python failo paleidimas vien iš komandinės eilutės.

Bet lengvesniam programavimui siūloma įsiinstaliuoti VS Code redaktorių, nes jis turi darbui su Python skirtą papildymą. Redaktorių galima atsisiųsti iš <https://code.visualstudio.com>. Įsidiegus šią

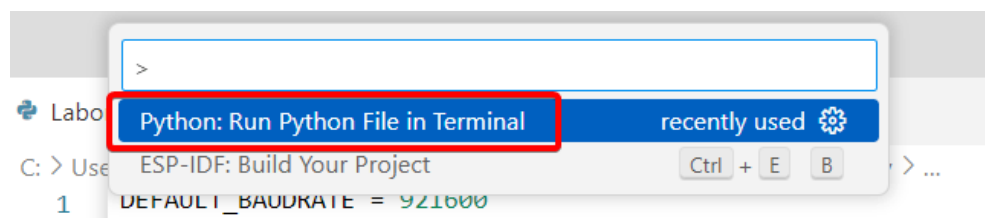
programinę įrangą, reikia nueiti į papildymų skyrelį (angl. *Extensions*) ir parsisiųsti Microsoft įmonės Python paketą (žr. 4 pav.).



4 pav. VS Code Python papildymo įdiegimo žingsniai.

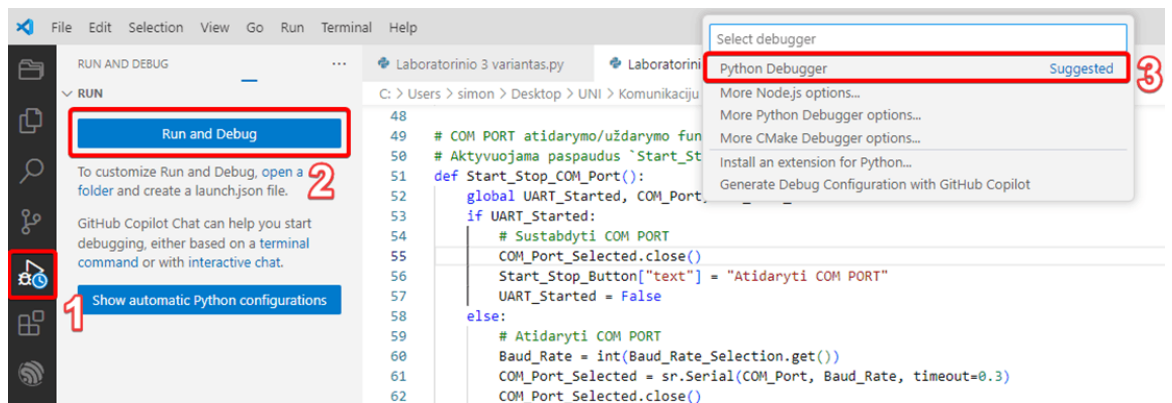
Šis redaktorius ir papildymas leis:

1. Lengviau redaguoti tekstą;
2. Lengviau paleisti Python programos failą. Ant klaviatūros paspaudus „F1“ mygtuką - iššoka komandų langas, kuriame pasirinkus „Run Python File in Terminal“ (žr. 5 pav.) programa automatiškai paleidžiama su visais reikalingais argumentais.

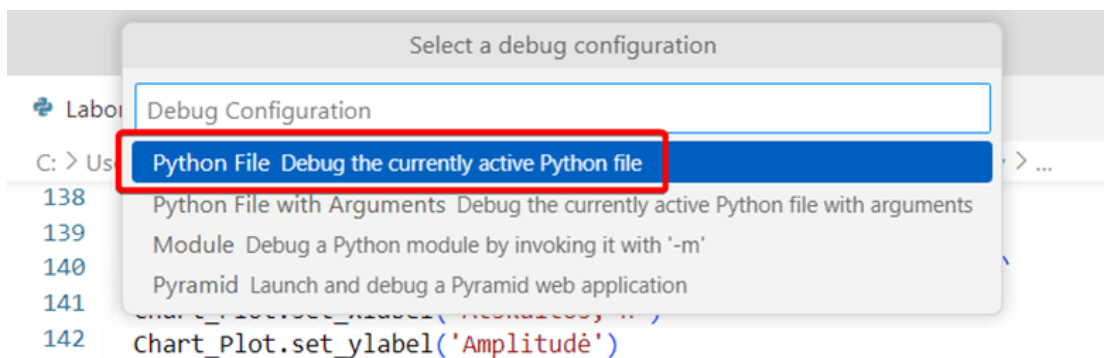


5 pav. Python programos paleidimo iš VS Code aplinkos langas.

3. Derinti kodą realiu laiku – leisti kodą eilutė po eilutės su stabdymo taškais (angl. *breakpoints*), peržiūrėti atmintį. Šiam režimui paleisti spaudžiami mygtukai ekrano kairėje (žr. 6 pav. ir 7 pav.).



6 pav. VS Code kodo derinimo režimo įjungimo pirmi trys žingsniai.



7 pav. VS Code kodo derinimo režimo įjungimo paskutinis žingsnis.

Išinstaliavus Python ir VS Code programinius paketus, galima parsisiųsti Python bibliotekas, reikalingas šiam laboratoriniam darbui. Python bibliotekas galima parsisiųsti suvedus komandas į komandų terminalo langą. „Command Prompt“ komandos su rekomenduojamomis ir pavyzdiniame projekte naudojamomis Python bibliotekomis:

- python -m pip install -U pip
- python -m pip install -U matplotlib
- python -m pip install -U pyserial
- python -m pip install -U tk

```
Command Prompt
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\simon>python -m pip install -U matplotlib
WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\simon\AppData\Roaming\Python\Python312\site-packages)
WARNING: Ignoring invalid distribution ~ip (C:\Python312\Lib\site-packages)
Collecting matplotlib
  Downloading matplotlib-3.10.1-cp312-cp312-win_amd64.whl.metadata (11 kB)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\simon\appdata\roaming\python\python312\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\simon\appdata\roaming\python\python312\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\python312\lib\site-packages (from matplotlib) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\python312\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in c:\python312\lib\site-packages (from matplotlib) (2.2.2)
Requirement already satisfied: packaging>=20.0 in c:\users\simon\appdata\roaming\python\python312\site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in c:\python312\lib\site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\python312\lib\site-packages (from matplotlib) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\simon\appdata\roaming\python\python312\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\simon\appdata\roaming\python\python312\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading matplotlib-3.10.1-cp312-cp312-win_amd64.whl (8.1 MB)
----- 8.1/8.1 MB 2.1 MB/s eta 0:00:00
WARNING: Ignoring invalid distribution ~atplotlib (C:\Users\simon\AppData\Roaming\Python\Python312\site-packages)
WARNING: Ignoring invalid distribution ~ip (C:\Python312\Lib\site-packages)
Installing collected packages: matplotlib
WARNING: Ignoring invalid distribution ~ip (C:\Python312\Lib\site-packages)
Successfully installed matplotlib-3.10.1
```

8 pav. Komandų terminalo lange instaliuojama „matplotlib“ biblioteka.

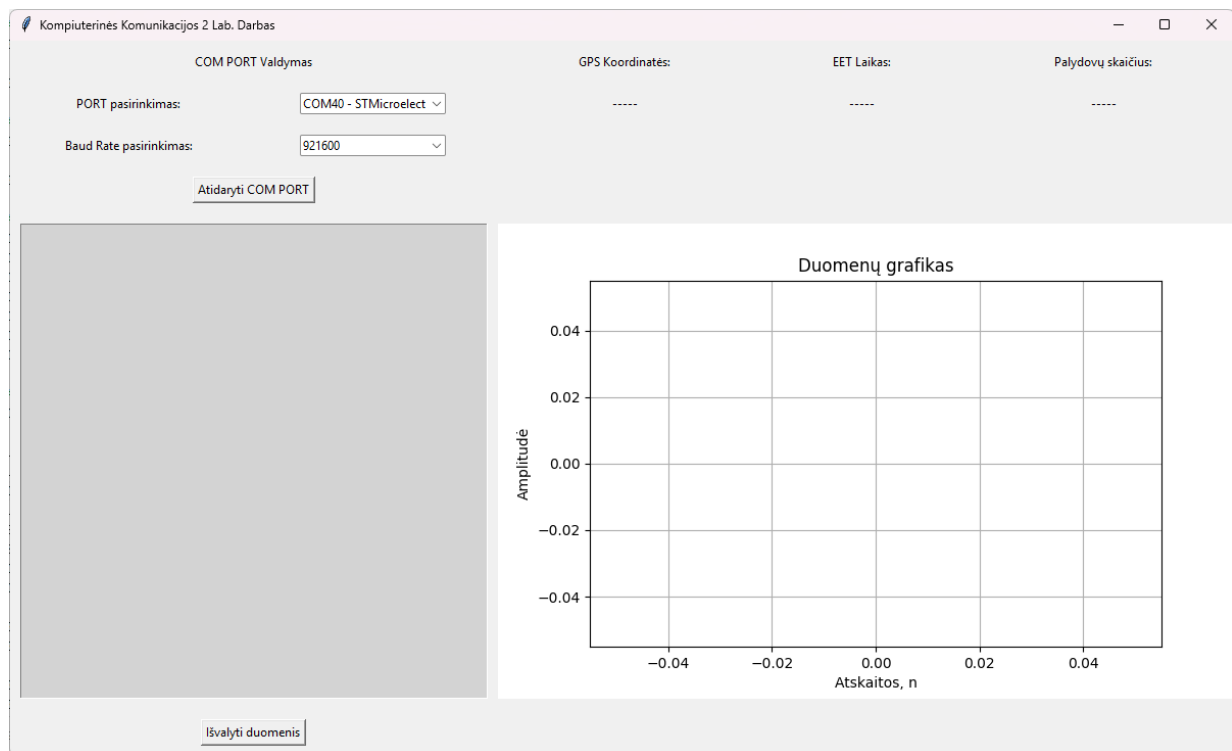
2. Grafinė sąsaja Python aplinkoje

Grafinė sąsaja Python kalboje gali būti sukurta remiantis keliomis bibliotekomis:

- Tkinter;
- PyQt;
- wxPython ir t.t.

Šiame laboratoriniame darbe bus remiamasi pirmąja biblioteka. Tkinter jau yra integruota atsisiunčiant Python, todėl yra dažniausiai sutinkama mažų grafinių aplikacijų kūrimo. Oficiali jos implementacija neturi atskiro lango grafinių elementų sukūrimui ir išdėstymui, todėl visa konfigūracija vykdoma tik programiniu kodu.

Žemiau 9 pav. yra pateikiamas Python programinėje aplinkoje sukurtas pavyzdys, kaip galėtų atrodyti grafinė vartotojo sąsaja duomenų nuskaitymui iš laboratorinio darbo maketo.



9 pav. Grafinės vartotojo sąsajos langas Python aplinkoje.

Grafinei vartotojo sąsajai sukurti pasitelkiamos naudojamos bibliotekos funkcijos:

1. *tk.Tk()* – grafinės sąsajos pagrindinis langas;
2. *tk.Label()* – Tekstinis laukelis;
3. *ttk.ComboBox()* – Pasirenkamo sąrašo laukelis;
4. *FigureCanvasTkAgg* – Jutiklių informacijos grafiko atvaizdavimas;
5. *tk.Button()* – Mygtukas.

3. Python grafinės sąsajos pagrindiniai funkcionalumai

Pateikiamame grafinės sąsajos pavyzdyje yra realizuotas minimalus funkcionalumas: prisijungimas prie COM PORT, gautų duomenų analizė bei jų atvaizdavimas tekstiniu ir grafiniu pavidalu. Šiame skyriuje pateikiami šių funkcijų aprašymai.

Programos kodas Python programose yra dažniausiai išdėstytas eilės tvarka (žr. 10 pav.):

1. **Pirma importuojamos** reikiamos bibliotekos;
2. **Sukuriamos globalios** konstantos ir kintamieji;
3. **Aprašomos funkcijos**;
4. **Inicializuojamas** grafinės sąsajos langas;
5. **Sukuriami individualūs** grafiniai elementai;
6. **Elementai išdėstomi** grafinės sąsajos lange;
7. **Paleidžiama** sukurta aplikacija – įjungiamas begalinis ciklas.

```
1 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
2 from matplotlib.figure import Figure
3 ...
4
5 # Konstantos
6 DISPLAY_CHART_AMOUNT = 10
7 DEFAULT_BAUDRATE = 921600
8 ...
9
10 # Funkcijos
11 # COM PORT aptarnavimo funkcija
12 def Display_UART_Data():
13     global COM_Port_Selected, UART_Started, chart_x_data, chart_y_data
14     if UART_Started and COM_Port_Selected.in_waiting > 0:
15         # Nuskaityta viena duomenų eilutė
16         COM_Port_Data = COM_Port_Selected.readline().decode('ascii').strip()
17     ...
18
19 # Grafinės vartotojo sąsajos kūrimas
20 # Pagrindinės aplikacijos langas
21 Application_Window = tk.Tk()
22 Application_Window.title('Kompiuterinės Komunikacijos 2 Lab. Darbas')
23 Application_Window.geometry("1200x700")
24 ...
25
26 # COM PORT valdymo elementai
27 UART_Label = tk.Label(Application_Window, text = "COM PORT Valdymas")
28 UART_COM_Port_Label = tk.Label(Application_Window, text = "PORT pasirinkimas:")
29 ...
30
31 # Grafinės vartotojo sąsajos elementų išdėstymas lentelės principu
32 UART_Label.grid(row=0, column=0, columnspan=2, padx=10, pady=10)
33 UART_COM_Port_Label.grid(row=1, column=0, padx=10, pady=10)
34 ...
35
36 # Aplikacijos atidarymas ir pagrindinės funkcijos iškvietimas
37 Application_Window.after(1, Display_UART_Data)
38 Application_Window.mainloop()
```

10 pav. Programos kodo išdėstymo pavyzdys.

3.1. COM PORT

COM PORT – prievadas skirtas UART duomenų nuskaitymui USB sąsaja.

3.1.1. Prievado pasirinkimas

COM PORT pagrindiniai parametrai – numeris bei greitis pasirenkami dvejais išskleidžiamo sąrašo elementais. Šie elementai su savo pradinėmis vertėmis sukuriami programos pradžioje (žr. 11 pav. ir 12 pav.)

```

104 # COM PORT valdymo elementai
105 UART_Label = tk.Label(Application_window, text = "COM PORT Valdymas")
106 UART_COM_Port_Label = tk.Label(Application_window, text = "PORT pasirinkimas:")
107 UART_COM_Baud_Rate_Label = tk.Label(Application_window, text = "Baud Rate pasirinkimas:")
108 # COM PORT pasirinkimo sąrašo laukas
109 COM_Port_Selection = ttk.Combobox(Application_window, state='readonly')
110 COM_Port_Selection.set("Pasirinkti COM port")
111 COM_Port_Selection.bind('<Button-1>', update_serial_port_values)
112 available_ports = serial.tools.list_ports.comports()
113 if available_ports:
114     COM_Port_Selection.set(available_ports[0])
115     Get_COM_Port(None)
116 COM_Port_Selection.bind('<<ComboboxSelected>>', Get_COM_Port)

```

11 pav. COM PORT numerio pasirinkimo laukelio sukūrimas.

```

123 # Baud rate pasirinkimo sąrašo laukas
124 Baud_Rate_Selection = ttk.Combobox(Application_window, state='readonly')
125 Baud_Rate_Selection['values'] = [9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600]
126 Baud_Rate_Selection.set(DEFAULT_BAUDRATE)

```

12 pav. COM PORT greičio pasirinkimo laukelio sukūrimas.

Sukuriant COM PORT numerio pasirinkimo sąrašą reikia atkreipti dėmesį, kad jo vertės gali kisti – prijungus ar atjungus prie kompiuterio įrenginį vertės turi automatiškai atsinaujinti ir neleisti vartotojui atidaryti atjungto prievado. Tam sukurama funkcija, kuri išskviečiama kiekvieną kartą kai sąrašas yra išskleidžiamas (žr. 13 pav.)

```

54 # Atnaušina visus aktyvius COM PORT rodomus išskleidžiamame sąrašė `COM_Port_Selection`
55 def update_serial_port_values(event):
56     COM_Port_Selection['values'] = serial.tools.list_ports.comports()
57
58 # COM PORT pasirinkimo funkcija
59 # Aktyvuojama pasirinkus COM PORT iš sąrašo
60 def Get_COM_Port(event):
61     global COM_Port_Selected, COM_Port
62     # Pasirinkto COM PORT numerio atrinkimas
63     COM_Port_Full_Name = COM_Port_Selection.get().split()
64     COM_Port = COM_Port_Full_Name[0]
65     # Aktyvuojami COM PORT valdymo mygtukai
66     Start_Stop_Button["state"] = "normal"
67     Start_Stop_Button["text"] = "Atidaryti COM PORT"

```

13 pav. COM PORT numerių pasirinkimo atnaujinimo funkcija.

3.1.2. Prievado atidarymas ir uždarymas

Sukuriamas mygtukas prievado uždarymui/atidarymui atlikti (žr. 14 pav.)

```

117 Start_Stop_Button = tk.Button(Application_window, text = "Atidaryti COM PORT", command = lambda: Start_Stop_COM_Port(), state = "disabled")

```

14 pav. Mygtuko prievado atidarymui/uždarymui sukūrimas.

Paspaudus mygtuką išskviečiama funkcija „*Start_Stop_COM_Port*“ atidaranti arba uždaranti prievadą (žr. 15 pav.).

```

69 # COM PORT atidarymo/uždarymo funkcija
70 # Aktyvuojama paspaudus `Start_Stop_Button` mygtuką
71 def Start_Stop_COM_Port():
72     global UART_Started, COM_Port, COM_Port_Selected
73     if UART_Started:
74         # Sustabdyti COM PORT
75         COM_Port_Selected.close()
76         Start_Stop_Button["text"] = "Atidaryti COM PORT"
77         UART_Started = False
78     else:
79         # Atidaryti COM PORT
80         Baud_Rate = int(Baud_Rate_Selection.get())
81         COM_Port_Selected = sr.Serial(COM_Port, Baud_Rate, timeout=0.3)
82         COM_Port_Selected.close()
83         COM_Port_Selected.open()
84         Start_Stop_Button["text"] = "Uždaryti COM PORT"
85         UART_Started = True

```

15 pav. COM PORT atidarymo/uždarymo funkcija.

3.2. Grafiko sukūrimas

Grafikas, matomas dešinėje ekrano pusėje, sukuriamas pasinaudojant „matplotlib“ biblioteką. Grafiko kūrimo metu aprašomi ašių pavadinimai, grafiko antraštė (žr. 16 pav.).

```

138 # Grafikas
139 Chart_Figure = Figure()
140 Chart_Plot = Chart_Figure.add_subplot(111)
141 Chart_Plot.set_title('Duomenų grafikas')
142 Chart_Plot.set_xlabel('Atskaitos, n')
143 Chart_Plot.set_ylabel('Amplitudė')
144 Chart_Plot.grid()
145 Chart_Line = Chart_Plot.plot([],[])[0]
146 Chart_Canvas = FigureCanvasTkAgg(Chart_Figure, master=Application_window)

```

16 pav. Grafiko sukūrimo kodas.

Norint grafike atvaizduoti vertes sukuriami du globalūs masyvai skirti grafiko ašių duomenims saugoti – „chart_x_data“ ir „chart_y_data“. Papildžius šiuos kintamuosius 36 – 37-oje eilutėje visą grafiką reikia perpiešti iš naujo 43-oje eilutėje (žr. 17 pav.).

41 - 42-oje eilutėje vykdomas grafiko ašių verčių apribojimas:

- X ašis apribojama rodyti tik „DISPLAY_CHART_AMOUNT“ paskutinių gautų atskaitų kiekį (įprastai tik 10);
- Y ašis apribojama rodyti visa masyvo verčių diapazoną su +- 10 verčių tarpais iš apačios ir viršaus.

```

35 # Atnaujinami priimtų duomenų masyvai
36 chart_x_data.append(len(chart_x_data) + 1)
37 chart_y_data.append(value)
38
39 # Update the chart
40 Chart_Line.set_data(chart_x_data, chart_y_data)
41 Chart_Plot.set_xlim(max(1, len(chart_x_data) - DISPLAY_CHART_AMOUNT), len(chart_x_data))
42 Chart_Plot.set_ylim(min(chart_y_data) - 10, max(chart_y_data) + 10)
43 Chart_Canvas.draw()

```

17 pav. Grafiko masyvų papildymas ir jų atvaizdavimas.

3.3. UART duomenų priėmimas

Atidarius COM PORT kompiuterio tvarkyklės pradeda duomenų įrašymą į vidinius duomenų masyvus, todėl Python programoje tereikia juos nuskaityti ir bus turima visa reikalingą informacija.

Nuskaitymas vykdomas blokuojančiu režimu begaliniame cikle (žr. 18 pav.).

```
21 # Funkcijos
22 # COM PORT aptarnavimo funkcija
23 def Display_UART_Data():
24     global COM_Port_Selected, UART_Started, chart_x_data, chart_y_data
25     if UART_Started and COM_Port_Selected.in_waiting > 0:
26         # Nuskaityta viena duomenų eilutė
27         COM_Port_Data = COM_Port_Selected.readline().decode('ascii').strip()
28
29         # Tolesnis kodas skirtas pavyzdinės UART žinutės - "SR vertė:-55" apdoroti
30         # Duomenys atskiriami pagal ":" ženkla ir išsaugomi į masyvą
31         data_parts = COM_Port_Data.split(":")
32         # Kintamojo vertė bus visada antroje masyvo vietoje
33         value = int(data_parts[1])
34
35         # Atnaujinami priimtų duomenų masyvai
36         chart_x_data.append(len(chart_x_data) + 1)
37         chart_y_data.append(value)
38
39         # Update the chart
40         Chart_Line.set_data(chart_x_data, chart_y_data)
41         Chart_Plot.set_xlim(max(1, len(chart_x_data) - DISPLAY_CHART_AMOUNT), len(chart_x_data))
42         Chart_Plot.set_ylim(min(chart_y_data) - 10, max(chart_y_data) + 10)
43         Chart_Canvas.draw()
44
45         # Atvaizduoti neapdorotus duomenis tekstiniame laukelyje
46         COM_Port_UART_Data_Display.config(state='normal')
47         COM_Port_UART_Data_Display.insert(tk.END, COM_Port_Data + '\n')
48         COM_Port_UART_Data_Display.config(state='disabled')
49         COM_Port_UART_Data_Display.yview(tk.END)
50
51 # Iškviečiama ta pati funkcija iš naujo, sudaromas begalinis ciklas
52 Application_Window.after(1, Display_UART_Data)
```

18 pav. UART duomenų nuskaitymo ir apdorojimo begalinis ciklas.

Begalinis ciklas aprašomas funkcija „*Display_UART_Data*“, kuri pirmą kartą yra iškviečiama vienoje iš paskutinių kodo eilučių – 170-oje (žr. 19 pav.), o po to rekursyviai pakartotinai kviečiama pačioje funkcijos viduje 52-oje eilutėje.

Vienetas begalinio ciklo funkcijos kvietime indikuoja, kad ji bus aktyvuojama kas 1 ms.

```
169 # Aplikacijos atidarymas ir pagrindinės funkcijos iškvietimas
170 Application_Window.after(1, Display_UART_Data)
171 Application_Window.mainloop()
```

19 pav. Begalinio ciklo iškvietimas programos vykdymo gale.

3.4. UART duomenų apdorojimas

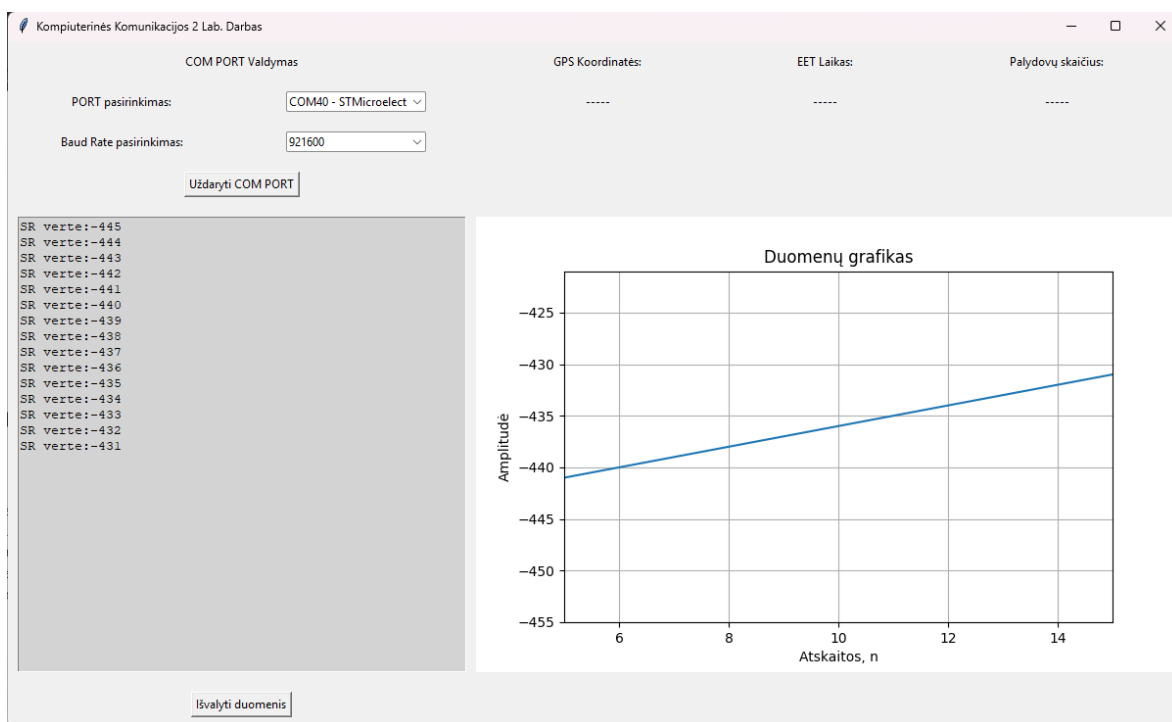
Priimami duomenys taip pat ir apdorojami, kadangi visų studentų siunčiamos žinutės skirsis dėl jutiklių bei vardų įvairovės, todėl šiame pavyzdyje pateikiamas duomenų ištraukimas iš paprastos žinutės su vienu kintamuoju.

Pavyzdyje pateikiama žinutė C kalboje turi formatą: „SR vertė:%ld“, kur %ld reiškia bet kokią skaičių su ženklu. Pilna žinutė atrodytų pvz.: „SR vertė:77“.

Dėl tokio formato paprasčiausia skaitmens vertę iš tekstinio laukelio yra nuskaityti žingsniais:

1. Teksto laukelis išskaidomas į atskiras dalis pagal dvitaškio ženklą – vienoje dalyje liks „SR vertė“, kitoje – ieškomas skaičius;
2. Turint išskaidytas dalis tekstinio masyvo pavidalu vertė bus visada masyvo indekso 1 vietoje.

Ši vertė yra patalpinama į bendrų duomenų masyvą „chart_x_data“ ir „chart_y_data“ – atvaizduojama grafike, bei užpildoma tekstiniam laukelyje (žr. 20 pav.).



20 pav. Grafinės vartotojo sąsajos langas su atvaizduojamais duomenimis Python aplinkoje.

Laboratorinio darbo metu reikės iš tekstinių duomenų ištraukti daugiau nei vieną kintamąjį – tam atlikti siūloma juos visus atskirti tam tikru ženklu. Pasinaudojus pavyzdinio kodo fragmentu, kelios skirtingos vertės bus skirtingose masyvo vietose, todėl jų nuskaitymas bus paprastas kaip skirtingų masyvo verčių panaudojimas.

3.5. Grafinės sąsajos elementų išdėstymas

Sukurtus grafinius elementus – mygtukus, grafikus, teksto laukelius, reikia išdėstyti programos lange. Pats programos langas su pradiniu dydžiu bei pavadinimu sukuriamas programos pradžioje (žr. 21 pav.)

```

94 # Grafinės vartotojo sąsajos kūrimas
95 # Pagrindinės aplikacijos langas
96 Application_window = tk.Tk()
97 Application_window.title('Kompiuterinės Komunikacijos 2 Lab. Darbas')
98 Application_window.geometry("1200x700")

```

21 pav. Grafinės sąsajos lango sukūrimo kodas.

Dizaino metu taip pat svarbu įvertinti, kad modernioms programoms svarbu atrodyti gražiai - programos elementai atitiktų programos lango dydį, su juo trauktųsi ir didintųsi.

Paprasčiausia tai atlikti su „tkinter“ bibliotekoje prieinama „grid“ funkcija, kuri turi galimybę elementus išdėstyti lentelės pavidalu. Ši lentelė visada prisitaikys ir bandys užpildyti visą programos langą.

Lentelės išdėstymą realizuoti reikalinga sukūrus individualų grafinį elementą jam priskirti vietą pakvietus „grid()“ funkciją su atitinkamais parametrais (žr. 22 pav.):

- *Row* – kurioje eilutėje turi būti elementas;
- *Column* – kuriame stulpelyje turi būti elementas;
- *Padx* ir *Pady* – koks atitraukimas nuo šalia esančių grafinių elementų;
- *Column/row span* – kiek eilučių ar stulpelių elementas užima. Svarbu didesniems elementams kaip grafikams ar tekstiniais laukams. Tai padeda užtikrinti jų dydį ir leidžia neištempti smulkesnių elementų kai jie yra išdėstyti šalia didžiųjų.

```

154 # Grafinės vartotojo sąsajos elementų išdėstymas lentelės principu
155 UART_Label.grid(row=0, column=0, columnspan=2, padx=10, pady=10)
156 UART_COM_Port_Label.grid(row=1, column=0, padx=10, pady=10)
157 COM_Port_Selection.grid(row=1, column=1, padx=10, pady=10)
158 UART_COM_Baud_Rate_Label.grid(row=2, column=0, padx=10, pady=10)
159 Baud_Rate_Selection.grid(row=2, column=1, padx=10, pady=10)
160 Start_Stop_Button.grid(row=3, column=0, columnspan=2, padx=10, pady=10)
161 COM_Port_UART_Data_Display.grid(row=4, column=0, columnspan=2, padx=10, pady=10, sticky="nsew")
162 Clear_Button.grid(row=5, column=0, columnspan=2, padx=10, pady=10)
163
164 GPS_Coordinates_Label.grid(row=0, column=2, padx=10, pady=10)
165 GPS_Coordinates_Value.grid(row=1, column=2, padx=10, pady=10)
166
167 GPS_Time_Label.grid(row=0, column=3, padx=10, pady=10)
168 GPS_Time_Value.grid(row=1, column=3, padx=10, pady=10)
169
170 GPS_Satellites_Label.grid(row=0, column=4, padx=10, pady=10)
171 GPS_Satellites_Value.grid(row=1, column=4, padx=10, pady=10)
172
173 Chart_Canvas.get_tk_widget().grid(row=4, column=2, columnspan=3, padx=0, pady=10, sticky="nsew")
174

```

22 pav. Grafinės sąsajos elementų išdėstymo lentelė pavyzdys.

Norint užtikrinti, kad lentelės stulpeliai ir eilutės dinamiškai užpildytų lango matmenis reikalinga nustatyti jiems svorius (žr. 23 pav.). Svarbu paminėti, kad nors svoris pritaikomas visiems lentelės stulpeliams, bet ne visoms eilutėms. Norint, kad programa nesuspaustų programos viršuje esančio teksto svoriai pritaikomi tik 4 eilutei – tempiant ekraną aukštį keičia tik teksto laukelis ir grafikas.

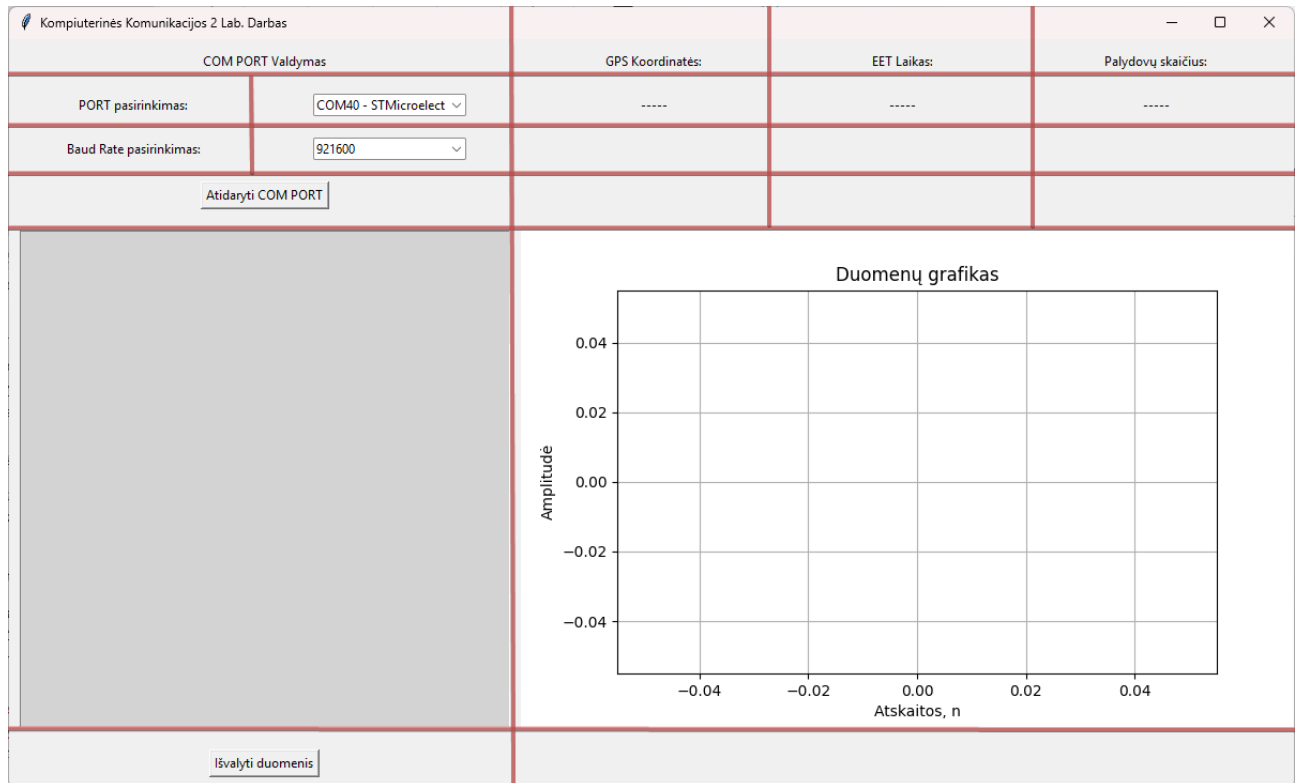
```

99     for i in range(4, 5):
100     |     Application_window.grid_rowconfigure(i, weight=1)
101     for i in range(5):
102     |     Application_window.grid_columnconfigure(i, weight=1)

```

23 pav. Lentelės išdėstymo svorių konfigūravimas.

Pavyzdinio kodo lentelė išdėstymą galima įsivaizduoti ekraną padalinus į blokelius pagal grafiniams elementams atliktą konfigūraciją, kaip pavaizduota 24 pav.



24 pav. Pavyzdinės grafinės sąsajos lentelės elementų išdėstymo blokų vizualizavimas.

Norint pridėti savų elementų galima panaudoti nenaudojamas eilučių 1 - 3 stulpelių 2 – 4 vietas, arba bandyti elementus išdėstyti už grafiko ribų (eilutė 5 arba stulpelis 5).

4. Laboratorinio darbo užduotis

1. Nuodugniai susipažinkite su laboratorinio darbo aprašymo 1-3 skyriuose pateikta medžiaga;
2. Įsidiekite Python programavimo kalbą ir VS Code redagavimo aplinką;
3. Asmeniniame kompiuteryje pasileiskite VS Code aplinką ir užsikraukite kurso Moodle puslapyje pateikiamą 2 laboratorinio darbo programinio kodo pavyzdį Python kalboje.
4. Priimkite pirmojo laboratorinio darbo duomenų paketą iš laboratorinio darbo maketo ir patikrinkite, ar paketas perduodamas teisingai naudojantis kuria nors komercine terminaline programa;
5. Realizuokite priimamo duomenų paketo iš laboratorinio darbo maketo per COM PORT funkcionalumą Python aplinkoje pasinaudodami kurso Moodle puslapyje pateiktu programinio kodo pavyzdžiu. Duomenų paketą atvaizduokite tekstiniame laukelyje;
6. Atvaizduokite individualaus varianto jutiklio duomenis Python aplinkoje grafiko pavidalu;
7. Tekstiniu pavidalu atvaizduokite GPS imtuvo koordinates, laiką ir palydovų skaičių. Su kiekvienu gautu GPS duomenų paketu atnaujinkite tekstinius laukelius su nauja gauta informacija;
8. Laboratorinio darbo ataskaitoje pateikite:
 - Išsikeltą darbo tikslą;
 - Atliktos darbo užduoties 4-7 žingsnių iliustracijas ir komentarus;
 - Laboratorinio darbo išvadas.

5. Savipatikros klausimai

1. Kaip atliekamas COM PORT konfigūravimas Python kalboje?
2. Kaip sudaromas sąrašo tipo laukelis su keliais pasirinkimais Python kalboje?
3. Kaip vykdomas nuoseklus duomenų nuskaitymas per UART sąsają Python aplinkoje?
4. Kokia yra Tkinter, PyQt, wxPython bibliotekų paskirtis?
5. Kaip atrenkamos FLOAT tipo vertės iš tekstinio duomenų paketo Python kalboje?
6. Kaip gali būti išskaidomas duomenų paketas į kelis masyvo elementus?
7. Kaip atvaizduojami jutiklio duomenys grafike naudojant Tkinter biblioteką?

Naudingos nuorodos

1. Python grafinės bibliotekos *tkinter* aprašas - <https://realpython.com/python-gui-tkinter/>
2. Python skaičių atrinkimas iš tekstinių duomenų - <https://www.geeksforgeeks.org/python-extract-numbers-from-string/>
3. *Matplotlib* bibliotekos grafikų naudojimas *tkinter* grafinėje sąsajoje - <https://www.tutorialspoint.com/how-can-we-run-matplotlib-in-tkinter>